

GPIO Driver Documentation

Driver Overview

GPIO stands for **General-Purpose Input/Output**. GPIO pins are used to communicate with external digital devices. A pin can be configured as an input to read signals from a switch or sensor, or as an output to control an LED, buzzer, relay, or other device.

1. Introduction

GPIO pins are digital interfaces available on most microcontrollers and embedded systems. Each GPIO pin can usually be configured independently as an input or output.

A GPIO driver provides software functions to:

- Configure a GPIO pin.
- Set a GPIO pin HIGH or LOW.
- Read the state of an input pin.
- Enable pull-up or pull-down resistors.
- Configure output type and drive strength.
- Enable GPIO interrupts.
- Release or reset a GPIO pin.

2. Objectives

The main objectives of a GPIO driver are:

- To initialize the GPIO peripheral.
- To configure individual pins.
- To read digital input signals.
- To generate digital output signals.
- To support interrupt-based GPIO operation.

- To provide safe and reusable functions.
- To prevent invalid access to GPIO pins.

3. GPIO Configuration Process

The `GPIO_Init()` API is responsible for configuring a GPIO pin.

The configuration sequence is:

- Enable GPIO peripheral clock.
- Configure pin mode.
- Configure output speed.
- Configure pull-up/pull-down settings.
- Configure output type.
- Configure alternate function if required.
- Configure EXTI and SYSCFG registers if interrupt mode is selected.

The driver automatically updates the required register fields and hides hardware-specific details from the application.

Interrupt Mode

A GPIO interrupt can be generated when:

- A rising edge occurs.
- A falling edge occurs.
- Either edge occurs.
- The input becomes HIGH.
- The input becomes LOW.

The driver supports GPIO interrupt generation through the EXTI peripheral.

When interrupt mode is selected:

- EXTI trigger registers (RTSR/FTSR) are configured.
- SYSCFG EXTICR registers map the GPIO port to the corresponding EXTI line.
- EXTI interrupt mask register (IMR) is configured.
- NVIC interrupt enable and priority configuration are handled separately using dedicated APIs.

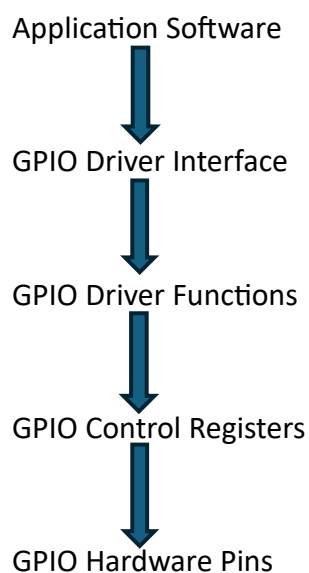
When an interrupt occurs:

- EXTI sets the pending bit.
- NVIC invokes the interrupt service routine.
- The application calls GPIO_IRQHandling().
- The driver clears the pending interrupt flag.

This abstraction allows the application to use interrupts without direct EXTI register access.

4. GPIO Driver Architecture

The GPIO driver is located between the application and the GPIO hardware.



5. Available APIs

1. GPIO_PeriClockControl()
2. GPIO_Init()
3. GPIO_DeInit()
4. GPIO_ReadFromInputPin()
5. GPIO_ReadFromInputPort()
6. GPIO_WriteToOutputPin()
7. GPIO_WriteToOutputPort()
8. GPIO_ToggleOutputPin()
9. GPIO_IRQInterruptConfig()

- 10. GPIO_IRQPriorityConfig()
- 11. GPIO_IRQHandling()

1. GPIO_PeriClockControl()

Purpose:

Enable or disable clock for a GPIO peripheral.

Abstraction Provided:

Hides RCC AHB1ENR register manipulation from the application.

Parameters:

- pGPIOx → GPIO peripheral base address
- EnorDi → ENABLE or DISABLE

2. GPIO_Init()

Purpose:

Initialize a GPIO pin according to user configuration.

Abstraction Provided:

Configures MODER, OTYPER, OSPEEDR, PUPDR, AFR, EXTI, and SYSCFG registers using a single API.

Parameters:

- pGPIOHandle → Pointer to GPIO handle structure

3. GPIO_DeInit()

Purpose:

Reset a GPIO peripheral to its default state.

Abstraction Provided:

Uses RCC reset registers internally and hides reset sequence details.

Parameters:

- pGPIOx → GPIO peripheral base address

4. *GPIO_ReadFromInputPin()*

Purpose:

Read logic level from a specific GPIO pin.

Abstraction Provided:

Returns pin state by internally reading the IDR register.

Returns:

- 0 → Logic Low
- 1 → Logic High

5. *GPIO_ReadFromInputPort()*

Purpose:

Read complete GPIO port state.

Abstraction Provided:

Returns 16-bit value from the GPIO input data register.

Returns:

- 16-bit port value

6. *GPIO_WriteToOutputPin()*

Purpose:

Set or reset a GPIO output pin.

Abstraction Provided:

Modifies the corresponding ODR register bit without exposing register operations.

Parameters:

- pGPIOx → GPIO peripheral base address
- PinNumber → GPIO pin number
- Value → GPIO_PIN_SET or GPIO_PIN_RESET

7. *GPIO_WriteToOutputPort()*

Purpose:

Write data to the entire GPIO port.

Abstraction Provided:

Updates all GPIO output pins through a single API call.

8. GPIO_ToggleOutputPin()

Purpose:

Toggle the state of a GPIO output pin.

Abstraction Provided:

Performs bit manipulation internally and hides register access.

9. GPIO_IRQInterruptConfig()

Purpose:

Enable or disable a GPIO interrupt in NVIC.

Abstraction Provided:

Handles NVIC ISER and ICER register programming.

Parameters:

- IRQNumber → Interrupt number
- EnorDi → ENABLE or DISABLE

10. GPIO_IRQPriorityConfig()

Purpose:

Configure interrupt priority.

Abstraction Provided:

Programs NVIC priority registers and hides priority bit positioning details.

Parameters:

- IRQNumber → Interrupt number
- IRQPriority → Priority value

11. GPIO_IRQHandling()

Purpose:

Clear pending GPIO interrupt.

Abstraction Provided:

Handles EXTI pending register access internally.

Parameters:

PinNumber → GPIO pin number

6. Abstraction Provided by the Driver

The GPIO driver hides the following hardware details from the application:

- RCC clock enable registers
- GPIO configuration registers
- Alternate function registers
- EXTI interrupt configuration registers
- SYSCFG EXTI mapping registers
- NVIC interrupt enable registers
- NVIC priority registers

The application only uses high-level APIs and configuration structures while the driver performs all required register manipulations internally.

7. Conclusion

The GPIO driver provides a clean abstraction layer over the STM32F401 GPIO peripheral. It simplifies GPIO configuration, pin control, alternate function setup, and interrupt management through a set of reusable APIs. By hiding low-level register operations, the driver improves code modularity, readability, and maintainability while still allowing efficient access to hardware resources.